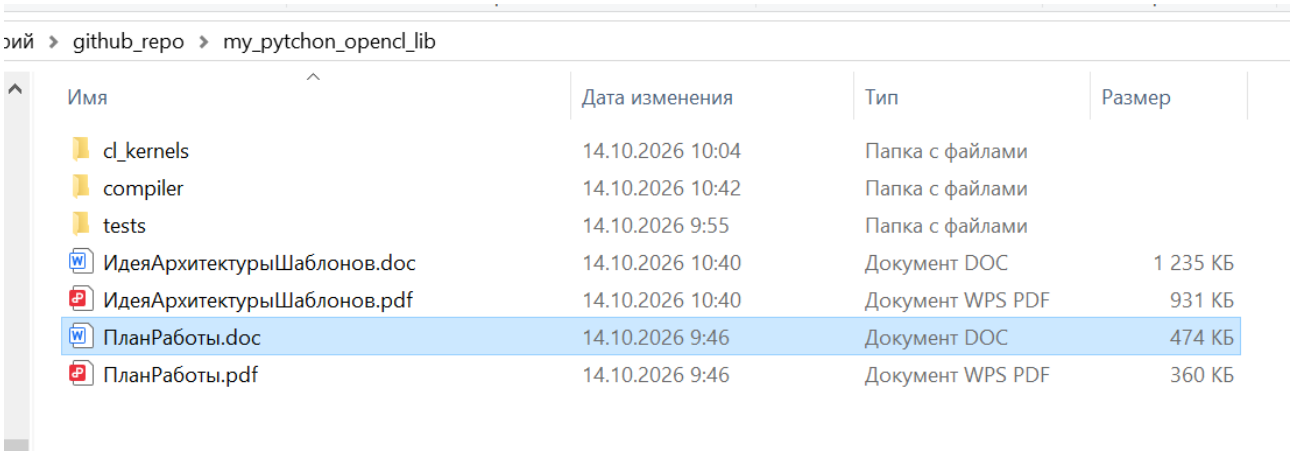
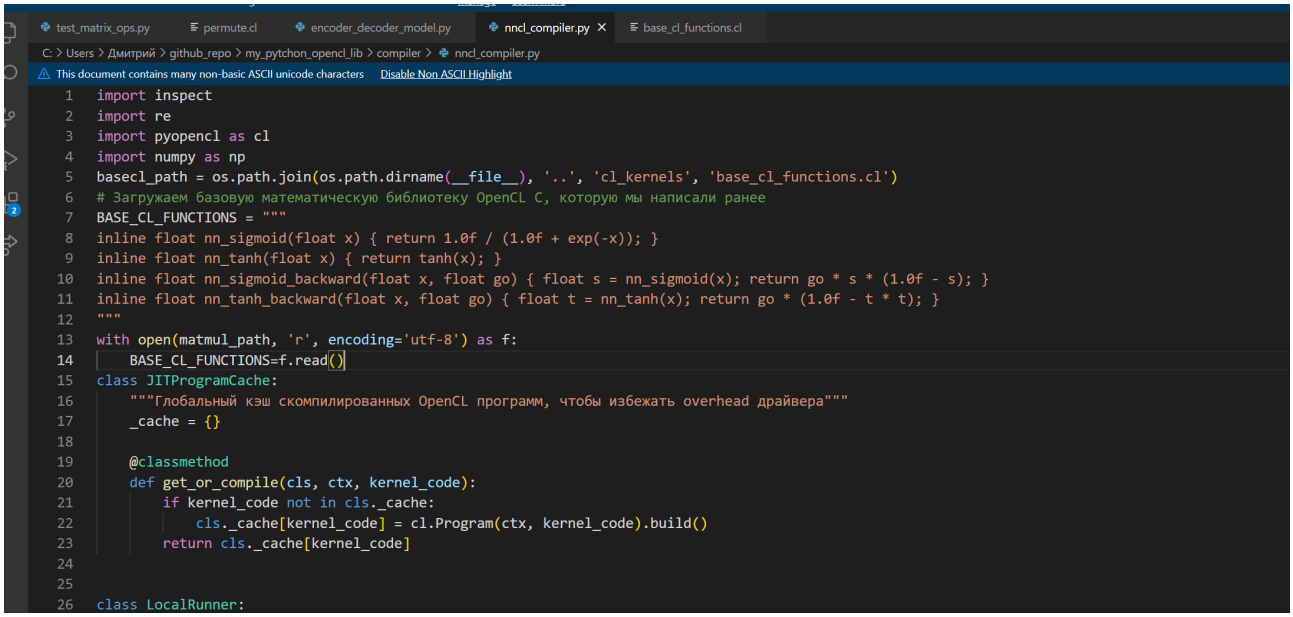
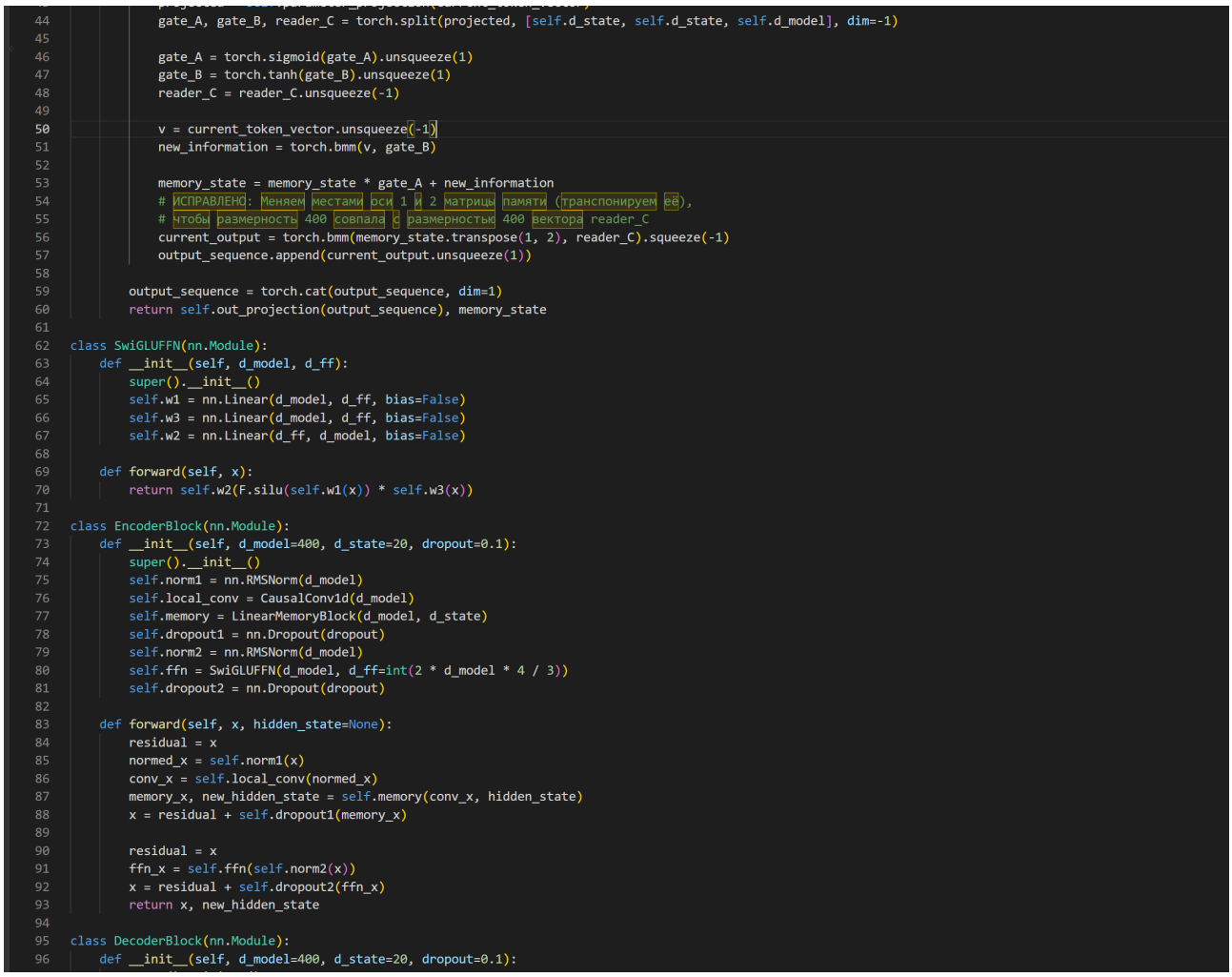
Вот текущая архитектура будущей библиотеки







```
            gate_A, gate_B, reader_C = torch.split(projected, [self.d_state, self.d_state, self.d_model], dim=-1)

            gate_A = torch.sigmoid(gate_A).unsqueeze(1)
            gate_B = torch.tanh(gate_B).unsqueeze(1)
            reader_C = reader_C.unsqueeze(-1)

            v = current_token_vector.unsqueeze(-1)
            new_information = torch.bmm(v, gate_B)

            memory_state = memory_state * gate_A + new_information
            # ИСПРАВЛЕНО: Меняем местами оси 1 и 2 матрицы памяти (транспонируем её),
            # чтобы размерность 400 совпала с размерностью 400 вектора reader_C
            current_output = torch.bmm(memory_state.transpose(1, 2), reader_C).squeeze(-1)
            output_sequence.append(current_output.unsqueeze(1))

        output_sequence = torch.cat(output_sequence, dim=1)
        return self.out_projection(output_sequence), memory_state

class SwiGLUFFN(nn.Module):
    def __init__(self, d_model, d_ff):
        super().__init__()
        self.w1 = nn.Linear(d_model, d_ff, bias=False)
        self.w3 = nn.Linear(d_model, d_ff, bias=False)
        self.w2 = nn.Linear(d_ff, d_model, bias=False)

    def forward(self, x):
        return self.w2(F.silu(self.w1(x)) * self.w3(x))

class EncoderBlock(nn.Module):
    def __init__(self, d_model=400, d_state=20, dropout=0.1):
        super().__init__()
        self.norm1 = nn.RMSNorm(d_model)
        self.local_conv = CausalConv1d(d_model)
        self.memory = LinearMemoryBlock(d_model, d_state)
        self.dropout1 = nn.Dropout(dropout)
        self.norm2 = nn.RMSNorm(d_model)
        self.ffn = SwiGLUFFN(d_model, d_ff=int(2 * d_model * 4 / 3))
        self.dropout2 = nn.Dropout(dropout)

    def forward(self, x, hidden_state=None):
        residual = x
        normed_x = self.norm1(x)
        conv_x = self.local_conv(normed_x)
        memory_x, new_hidden_state = self.memory(conv_x, hidden_state)
        x = residual + self.dropout1(memory_x)

        residual = x
        ffn_x = self.ffn(self.norm2(x))
        x = residual + self.dropout2(ffn_x)
        return x, new_hidden_state

class DecoderBlock(nn.Module):
    def __init__(self, d_model=400, d_state=20, dropout=0.1):
```

Нам надо теперь на opencl реализовать torch.norm1\2 , bmn (ксатит подскажи к какому уровню будут относиться эти функции (глобальный\локальный)?).
Также потом реализуем MultiheadAttention, pad (канальный паддинг)